Understanding the Recent Member API Changes

A line-by-line guide to the four latest commits for getTopUsers and getRestaurant

Prepared from the project history on the develop branch. The commits are explained in chronological order, including the original code, the later typo fixes, and what the current lines do.

Main idea: MongoDB field names are case-sensitive. In two filters, the code used capitalized field names that did not exist in the schema. Because this project enables Mongoose strictQuery, those unknown filter fields could be removed, leaving a query that matched an arbitrary first record.

The four commits covered

Order	Commit	What it changed
1	f6891e4 - feat: develop getTopUsers rest Api	Added the top-users service query, controller, and route. Its status filter initially had the casing error.
2	fffd57b - feat: create getRestaurant rest Api	Added the restaurant service query, controller, and route. Its type filter initially had the casing error.
3	47b2ae4 - fix: typo in Member.service.ts	Changed MemberType to memberType in the restaurant query.
4	3975608 - fix: typo fixed on line 110	Changed MemberStatus to memberStatus in top users, and changed the extra target string.

These are commits shown by git log. The word "push" is often used casually for saved project changes; a commit is the recorded change, while a push sends commits to a remote repository. This guide explains the four commits visible in the local history.

Before the line-by-line review

The project schema uses the field names `memberType` and `memberStatus`. Those exact spellings matter in MongoDB filters. The enum constants on the right side are values; they do not rename the database fields.

```
memberType: MemberType.RESTAURANT
memberStatus: MemberStatus.ACTIVE
```

The server also calls `mongoose.set("strictQuery", true)`. In Mongoose 6, strictQuery removes filter keys that are not in the schema. For example, a filter containing only an unknown key can become an empty filter `{}`. An empty filter can match any document, which explains why a query may return a regular member instead of a restaurant. This behavior is described in the Mongoose 6 guide.

```
Wrong field:   { MemberType: MemberType.RESTAURANT }
After strictQuery strips unknown key:  {}
Correct field: { memberType: MemberType.RESTAURANT }
```

A MongoDB document can have extra fields, but your Mongoose schema and your queries use the schema field names. Uppercase and lowercase letters are different characters.

Commit 1 - add getTopUsers

Commit: `f6891e4` - *feat: develop getTopUsers rest Api*. This commit added three connected pieces: a service query, a controller handler, and a route.

Service query - Member.service.ts

```
public async getTopUsers(): Promise<Member[]> {
  const result = await this.memberModel
    .find({ MemberStatus: MemberStatus.ACTIVE, memberPoints: { $gte: 1 } })
    .sort({ memberPoints: -1 })
    .limit(4)
    .exec();
  if (!result) throw new Errors(HttpCode.NOT_FOUND, Message.NO_DATA_FOUND);
  return result;
}
```

Code line	Purpose
`public async getTopUsers(): Promise<Member[]> {`	Declares an asynchronous service method that returns a list of member records. The angle-bracket type says each array item is a Member.
`const result = await this.memberModel`	Starts a Mongoose query against the member collection and waits for it to finish.
`.find({ MemberStatus: MemberStatus.ACTIVE, memberPoints: { $gte: 1 } })`	Finds records intended to be active and have at least one point. At this commit, `MemberStatus` was incorrectly capitalized, so it did not match the schema field.
`.sort({ memberPoints: -1 })`	Sorts by points descending: largest point totals first. The value -1 means descending.
`.limit(4)`	Returns at most four records.
`.exec();`	Executes the built query and resolves to the result array.
`if (!result) throw ...NO_DATA_FOUND;`	Intended as a not-found check. A Mongoose find query returns an array; even an empty array is truthy, so this condition does not catch zero matches.

Code line	Purpose
`return result;`	Returns the selected records to the controller.
`}`	Ends the method.

Controller and route

Code line	Purpose
`memberController.getTopUsers = async (req, res) => {`	Creates the HTTP handler. It is async because the service query takes time.
`console.log("getTopUsers");`	Prints a diagnostic message in the server terminal when the handler runs.
`const result = await memberService.getTopUsers();`	Calls the service and waits for the top-member array.
`res.status(HttpCode.OK).json(result)`	Sends HTTP 200 and serializes the result array as JSON.
`catch (err) { ... }`	Handles failures. An Errors instance uses its own status code; other errors use the standard server error response.
`router.get("/member/top-users", memberController.getTopUsers);`	Makes the handler available for GET requests at /member/top-users.

What needed correction later: the query used `MemberStatus` rather than the schema field `memberStatus`. With strictQuery enabled, that status constraint could be stripped, so blocked or deleted members with points could pass the remaining points filter.

Commit 2 - add getRestaurant

Commit: `fffd57b` - *feat: create getRestaurant rest Api*. It added a service lookup, controller, and route.

Original service query from this commit

```
public async getRestaurant(): Promise<Member> {
  const result = await this.memberModel
    .findOne({ MemberType: MemberType.RESTAURANT })
    .lean()
    .exec();
  result.target = "Test";
  if (!result) throw new Errors(HttpCode.NOT_FOUND, Message.NO_DATA_FOUND);
  return result;
}
```

Code line	Purpose
`public async getRestaurant(): Promise<Member> {`	Declares a service method that should return one member record, specifically the restaurant account.
`const result = await this.memberModel`	Starts a query against the member collection.
`.findOne({ MemberType: MemberType.RESTAURANT })`	Intended to select the restaurant. The filter key was wrong: schema uses lowercase `memberType`.
`.lean()`	Asks Mongoose to return a plain JavaScript object instead of a full Mongoose document.
`.exec();`	Executes the findOne query. It returns a document object or null if no match exists.
`result.target = "Test";`	Adds an arbitrary property named `target` to the returned plain object. This is not part of selecting a restaurant; it appears in the JSON response and is likely leftover test data.
`if (!result) throw ...NO_DATA_FOUND;`	Intended to report no restaurant found. In the original order, the previous target assignment ran first, so null would cause a TypeError before this intended error.
`return result;`	Returns the selected object to the controller.
`}`	Ends the service method.

Controller and route

Code line	Purpose
`memberController.getRestaurant = async (req, res) => {`	Creates the Express handler for the restaurant lookup.
`console.log("getRestaurant");`	Writes a diagnostic log when the endpoint is called.
`const result = await memberService.getRestaurant();`	Calls the service method and waits for the restaurant record.
`res.status(HttpCode.OK).json(result);`	Returns HTTP 200 and the record as JSON.
`catch (err) { ... }`	Logs failures and returns the custom error status when available, otherwise the standard error.
`router.get("/member/restaurant", memberController.getRestaurant)`	Registers the endpoint for GET /member/restaurant.

Because the original filter key was unknown, strictQuery could remove it and leave an empty filter. Then findOne could return the first member document, which may be a regular USER.

Commit 3 - correct the restaurant field name

Commit: `47b2ae4` - the commit message says there was a typo around line 25 of Member.service.ts. The functional change was:

```
Before: .findOne({ MemberType: MemberType.RESTAURANT })
After:  .findOne({ memberType: MemberType.RESTAURANT })
```

Code line	Purpose
`memberType`	Matches the exact field name in the schema.
`MemberType.RESTAURANT`	Uses the enum value "RESTAURANT" as the value to match. The enum constant itself may be capitalized; that is separate from the database field name.
`.lean().exec()`	Keeps the query execution and plain-object response behavior unchanged.
`result.target = "Test";`	This line was not removed by this fix. It remained an extra field on the output object.
`if (!result) ...`	Still checks whether no restaurant exists, but the target assignment is before it in the checked-in code.

This fix addresses the instructor complaint about getting an ordinary member: the query now filters by the real type field. It does not remove the extra target property or fix the null-check ordering.

Commit 4 - correct the top-users status field

Commit: `3975608` - *fix: typo fixed on line 110*. It includes two changes in the current history: the status filter correction and a change to the target test string.

```
Before: .find({ MemberStatus: MemberStatus.ACTIVE, memberPoints: { $gte: 1 } })
After:  .find({ memberStatus: MemberStatus.ACTIVE, memberPoints: { $gte: 1 } })
```

Code line	Purpose
`memberStatus: MemberStatus.ACTIVE`	Now uses the schema field name, so the query requires an active member. Blocked and deleted records are excluded.
`memberPoints: { $gte: 1 }`	Keeps only members with one or more points.
`.sort({ memberPoints: -1 })`	Keeps highest point totals first.
`.limit(4)`	Keeps the output to the top four matching records.
`result.target = "shungashunay";`	The latest commit changed the extra target property string. It still does not help find a restaurant and may be included in the endpoint JSON response.

The filter correction is what fixes blocked/deleted members appearing in getTopUsers. It does not add a memberType USER filter, so an active restaurant with at least one point would also qualify unless the data model prevents that.

Current behavior and takeaways

Code line	Purpose
`GET /member/restaurant`	Calls getRestaurant, which now queries memberType equal to RESTAURANT.
`GET /member/top-users`	Calls getTopUsers, which now filters memberStatus equal to ACTIVE and memberPoints greater than or equal to 1; sorts and limits to four.
`result.target = "shungashunay"`	Is still present in current Member.service.ts. It adds a non-schema property to the restaurant response. If there is no matching restaurant, this assignment can throw before the custom 404 error is reached.
`if (!result) for an array`	A find() result is always an array. To detect no top users, the appropriate condition would check result.length === 0. This is an observation, not a code change made for this explanation.

Reliable way to avoid this class of bug: copy database field names from the schema exactly. For a query filter, compare the left-hand key (database field) with the schema; the right-hand value (enum constant) can have a different capitalization.

Useful project references

Project history: commits f6891e4, fffd57b, 47b2ae4, and 3975608 on develop.

Current code locations: src/models/Member.service.ts (getRestaurant around lines 23-30; getTopUsers around lines 108-115), src/controllers/member.controller.ts (getRestaurant around lines 19-31; getTopUsers around lines 157-168), and src/router.ts (routes around lines 7 and 28).

Mongoose reference: Mongoose 6 Guide, "strictQuery" option - https://mongoosejs.com/docs/6.x/docs/guide.html